

Application: Floyd's Algorithm vs Warshall's Algorithm (DP Comparison)

CO4 AT3

CSA0619 – DESIGN AND ANALYSIS OF ALGORITHMS

Submitted by

M Yasodha Krishna (192472100)

Under the Supervision of

Dr. F. Mary Harin Fernandez

ABSTRACT

Graph-based systems are widely used in transportation networks, social networks, computer networks, and communication systems. Two important Dynamic Programming algorithms used for graph analysis are Floyd's Algorithm and Warshall's Algorithm. Floyd's Algorithm computes the shortest paths between every pair of vertices in a weighted graph, whereas Warshall's Algorithm determines the transitive closure or reachability between all pairs of vertices. Both algorithms use a similar Dynamic Programming formulation by progressively considering intermediate vertices. This report compares the objectives, outputs, implementation methods, time complexity, scalability, and applications of Floyd's and Warshall's algorithms. The results demonstrate how Dynamic Programming can efficiently solve important graph problems using iterative matrix-based computation.

INTRODUCTION

Graphs are commonly used to represent relationships between objects. For example, vertices can represent cities, computers, users, or locations, while edges represent roads, communication links, or relationships.

In graph problems, it is often necessary to determine whether two vertices are connected or to find the shortest path between them. These problems can be solved efficiently using Dynamic Programming algorithms.

Floyd's Algorithm, also called the Floyd-Warshall shortest path algorithm, calculates the shortest distance between every pair of vertices in a weighted graph.

Warshall's Algorithm calculates the transitive closure of a graph, determining whether a path exists between every pair of vertices.

Although their objectives are different, both algorithms use a similar Dynamic Programming approach by repeatedly updating a matrix while considering each vertex as an intermediate vertex.

PROBLEM STATEMENT

Given a graph, compute all-pairs shortest paths using Floyd's Algorithm and compute the transitive closure using Warshall's Algorithm.

The objective is to compare the outputs and objectives of both algorithms, analyze their similarities in Dynamic Programming formulation, evaluate their time complexity, discuss scalability, and generate insights into graph-based Dynamic Programming algorithms.

OBJECTIVES

- To understand graph-based Dynamic Programming algorithms.
- To implement Floyd's Algorithm for all-pairs shortest paths.

- To implement Warshall's Algorithm for transitive closure.
- To compare the objectives of both algorithms.
- To analyze similarities in their Dynamic Programming formulation.
- To compare the outputs generated by both algorithms.
- To evaluate time and space complexity.
- To discuss scalability for large graphs.

SYSTEM DESCRIPTION

The proposed system accepts a graph represented using an adjacency matrix.

For Floyd's Algorithm, the matrix contains edge weights. If there is no direct connection between two vertices, the value is represented as infinity.

For Warshall's Algorithm, the matrix contains binary values:

- 1 represents that a connection exists.
- 0 represents that no direct connection exists.

Both algorithms repeatedly consider every vertex as an intermediate vertex and update the corresponding matrix.

The final output of Floyd's Algorithm is a matrix containing the shortest distances between all pairs of vertices.

The final output of Warshall's Algorithm is a reachability matrix showing whether a path exists between every pair of vertices.

FLOYD'S ALGORITHM

Floyd's Algorithm finds the shortest path between every pair of vertices in a

weighted graph.

The algorithm checks whether using an intermediate vertex provides a shorter path.

The Dynamic Programming formula is:

$$D[i][j] = \min(D[i][j], D[i][k] + D[k][j])$$

Where:

- i = source vertex
- j = destination vertex
- k = intermediate vertex

The algorithm gradually considers all vertices as possible intermediate vertices.

WARSHALL'S ALGORITHM

Warshall's Algorithm finds the transitive closure of a directed graph.

It determines whether a path exists between every pair of vertices.

The Dynamic Programming formula is:

$$R[i][j] = R[i][j] \text{ OR } (R[i][k] \text{ AND } R[k][j])$$

Where:

- $R[i][j]$ indicates whether vertex j is reachable from vertex i .
- k is the intermediate vertex.

The algorithm gradually identifies indirect connections between vertices.

DYNAMIC PROGRAMMING APPROACH

Both algorithms follow a similar Dynamic Programming strategy.

The process consists of the following steps:

Step 1: Initialization

Create an adjacency or distance matrix.

Step 2: Intermediate Vertex Selection

Consider each vertex one by one as an intermediate vertex.

Step 3: Matrix Update

Update the matrix based on whether the intermediate vertex improves the result.

Step 4: Final Result

After all vertices are considered, the matrix contains the final solution.

The major difference is the operation used:

Floyd's Algorithm

Uses:

Minimum + Addition

Warshall's Algorithm

Uses:

OR + AND

SIMILARITIES BETWEEN FLOYD'S AND WARSHALL'S ALGORITHMS

Both algorithms:

- Use Dynamic Programming.

- Work with adjacency matrices.
- Consider every vertex as an intermediate vertex.
- Use three nested loops.
- Solve all-pairs graph problems.
- Have the same time complexity.
- Update the solution iteratively.
- Are suitable for dense graphs.

The main difference is the type of problem they solve.

ALGORITHM

FLOYD'S ALGORITHM

Step 1:

Start.

Step 2:

Read the weighted adjacency matrix.

Step 3:

Consider each vertex as an intermediate vertex.

Step 4:

Compare the existing distance with the distance through the intermediate vertex.

Step 5:

Update the distance if a shorter path is found.

Step 6:

Repeat for all vertices.

Step 7:

Display the shortest path matrix.

Step 8:

Stop.

WARSHALL'S ALGORITHM

Step 1:

Start.

Step 2:

Read the adjacency matrix.

Step 3:

Consider each vertex as an intermediate vertex.

Step 4:

Check whether a path exists through the intermediate vertex.

Step 5:

Update the reachability matrix.

Step 6:

Repeat for all vertices.

Step 7:

Display the transitive closure matrix.

Step 8:

Stop.

IMPLEMENTATION

Q19 - Floyd's Algorithm vs Warshall's Algorithm

Dynamic Programming Comparison

INF = 99999

Floyd's Algorithm

def floyd_algorithm(graph):

n = len(graph)

distance = []

for row in graph:

distance.append(row.copy())

for k in range(n):


```
for i in range(n):
```

```
    for j in range(n):
```

```
        if distance[i][k] + distance[k][j] < distance[i][j]:
```

```
            distance[i][j] = distance[i][k] + distance[k][j]
```

```
return distance
```

```
# Warshall's Algorithm
```

```
def warshall_algorithm(graph):
```

```
    n = len(graph)
```

```
    reachability = []
```

```
    for row in graph:
```

```
        reachability.append(row.copy())
```

```
    for k in range(n):
```

```
for i in range(n):
```

```
    for j in range(n):
```

```
        reachability[i][j] = (
```

```
            reachability[i][j]
```

```
            or
```

```
            (reachability[i][k] and reachability[k][j])
```

```
        )
```

```
    return reachability
```

```
# Function to print matrix
```

```
def print_matrix(matrix):
```

```
    for row in matrix:
```

```
        for value in row:
```

```
            if value == INF:
```

```
print("INF", end="\t")
```

```
else:
```

```
    print(value, end="\t")
```

```
print()
```

```
def main():
```

```
    print("=" * 65)
```

```
    print("FLOYD'S ALGORITHM VS WARSHALL'S ALGORITHM")
```

```
    print("DYNAMIC PROGRAMMING COMPARISON")
```

```
    print("=" * 65)
```

```
# Weighted Graph for Floyd's Algorithm
```

```
weighted_graph = [
```

```
    [0, 3, INF, 7],
```

```
[8, 0, 2, INF],

[5, INF, 0, 1],

[2, INF, INF, 0]

]

print("\nWEIGHTED ADJACENCY MATRIX")

print_matrix(weighted_graph)

# Floyd's Algorithm

shortest_paths = floyd_algorithm(weighted_graph)

print("\nSHORTEST PATH MATRIX USING FLOYD'S ALGORITHM")

print_matrix(shortest_paths)
```

```
# Graph for Warshall's Algorithm
```

```
adjacency_graph = [
```

```
    [1, 1, 0, 1],
```

```
    [1, 1, 1, 0],
```

```
    [1, 0, 1, 1],
```

```
    [1, 0, 0, 1]
```

```
]
```

```
print("\nADJACENCY MATRIX")
```

```
print_matrix(adjacency_graph)
```

```
# Warshall's Algorithm
```

```
transitive_closure = warshall_algorithm(adjacency_graph)
```

```
print("\nTRANSITIVE CLOSURE USING WARSHALL'S ALGORITHM")
```

```
print_matrix(transitive_closure)
```

```
print("\n" + "=" * 65)
```

```
print("COMPLEXITY ANALYSIS")
```

```
print("=" * 65)
```

```
print("Floyd's Algorithm Time Complexity :  $O(n^3)$ ")
```

```
print("Warshall's Algorithm Time Complexity :  $O(n^3)$ ")
```

```
print("Space Complexity :  $O(n^2)$ ")
```

```
print("=" * 65)
```

```
main()
```

SAMPLE OUTPUT

```
=====
=====
```

FLOYD'S ALGORITHM VS WARSHALL'S ALGORITHM

DYNAMIC PROGRAMMING COMPARISON

```
=====
=====
```

WEIGHTED ADJACENCY MATRIX

0	3	INF	7
8	0	2	INF
5	INF	0	1
2	INF	INF	0

SHORTEST PATH MATRIX USING FLOYD'S ALGORITHM

0	3	5	6
5	0	2	3
3	6	0	1
2	5	7	0

ADJACENCY MATRIX

1	1	0	1
1	1	1	0
1	0	1	1
1	0	0	1

TRANSITIVE CLOSURE USING WARSHALL'S ALGORITHM

1	1	1	1
1	1	1	1
1	1	1	1
1	1	1	1

=====

=====

COMPLEXITY ANALYSIS

=====

=====

Floyd's Algorithm Time Complexity : $O(n^3)$

Warshall's Algorithm Time Complexity : $O(n^3)$

Space Complexity : $O(n^2)$

=====

=====

OUTPUT SCREENSHOT

```

>>> Trace execution completed successfully.

=====
=====
FLOYD'S ALGORITHM VS WARSHALL'S ALGORITHM
DYNAMIC PROGRAMMING COMPARISON
=====

WEIGHTED ADJACENCY MATRIX
0      3      INF      7
8      0      2      INF
5      INF      0      1
2      INF      INF      0

SHORTEST PATH MATRIX USING FLOYD'S ALGORITHM
0      3      5      6
5      0      2      3
3      6      0      1
2      5      7      0

ADJACENCY MATRIX
1      1      0      1
1      1      1      0
1      0      1      1
1      0      0      1

TRANSITIVE CLOSURE USING WARSHALL'S ALGORITHM
1      1      1      1
1      1      1      1
1      1      1      1
1      1      1      1

=====
COMPLEXITY ANALYSIS
=====
Floyd's Algorithm Time Complexity      : O(n^3)
Warshall's Algorithm Time Complexity   : O(n^3)
Space Complexity                        : O(n^2)
=====
>>>

```

COMPARISON OF FLOYD'S AND WARSHALL'S ALGORITHMS

Parameter	Floyd's Algorithm	Warshall's Algorithm
Main Objective	Find shortest paths	Find reachability
Graph Type	Weighted graph	Directed/Unweighted graph
Output	Distance matrix	Transitive closure matrix

Parameter	Floyd's Algorithm	Warshall's Algorithm
DP Operation	Min + Addition	OR + AND
Time Complexity	$O(n^3)$	$O(n^3)$
Space Complexity	$O(n^2)$	$O(n^2)$
Intermediate Vertices	Yes	Yes
Matrix Based	Yes	Yes
Suitable For	Shortest path problems	Connectivity problems

COMPLEXITY ANALYSIS

Both Floyd's and Warshall's algorithms use three nested loops.

Therefore:

Time Complexity

$$O(n^3)$$

Space Complexity

$$O(n^2)$$

The algorithms require a matrix to store information for all pairs of vertices.

For small and medium-sized dense graphs, these algorithms are practical. However, as the number of vertices increases significantly, the cubic time complexity becomes computationally expensive.

SCALABILITY ANALYSIS

The scalability of both algorithms depends on the number of vertices.

For a graph with n vertices, approximately n^3 operations are required.

Small Graphs

Both algorithms perform efficiently.

Medium-Sized Graphs

The algorithms are still practical, especially for dense graphs.

Large Graphs

The cubic time complexity can result in high execution time.

Very Large Sparse Graphs

Alternative algorithms may be more efficient depending on the problem.

For example:

- Dijkstra's Algorithm can be useful for shortest paths in certain graphs.
- BFS/DFS can be used for reachability problems.

Therefore, Floyd's and Warshall's algorithms are generally more suitable when all-pairs information is required and the graph is not extremely large.

RESULT ANALYSIS

The implementation successfully applies both Floyd's Algorithm and Warshall's Algorithm to graph-based problems.

Floyd's Algorithm Result

The algorithm calculates the shortest distance between every pair of vertices. For example, if a direct path is expensive or unavailable, Floyd's Algorithm can identify

a shorter indirect path through intermediate vertices.

Warshall's Algorithm Result

The algorithm determines whether every vertex can reach every other vertex. In the given graph, the final transitive closure shows that all vertices are reachable from one another.

Dynamic Programming Insight

Both algorithms use the same fundamental idea:

At every stage, determine whether allowing a new intermediate vertex improves the existing solution.

The difference is in the definition of improvement:

- Floyd checks for a smaller distance.
- Warshall checks for a new reachable path.

GRAPH-BASED DP INSIGHTS

The comparison between Floyd's and Warshall's algorithms provides several important insights about Dynamic Programming:

1. Same Structure, Different Operations

Both algorithms have almost identical loop structures but solve different problems.

2. Intermediate States Are Important

Each iteration introduces one additional vertex as a possible intermediate point.

3. Matrix Representation Is Effective

Dynamic Programming states can be efficiently stored and updated using matrices.

4. General DP Principle

A complex problem can be solved by combining solutions to smaller subproblems.

5. Different Mathematical Operations

Dynamic Programming algorithms can use different operations depending on the objective:

- Minimum for optimization problems.
- Boolean operations for decision problems.

APPLICATIONS

Floyd's Algorithm

- Road and transportation networks.
- Network routing.
- Airline route planning.
- Robot navigation.
- Communication networks.
- GPS and path planning systems.

Warshall's Algorithm

- Social network connectivity.
- Website link analysis.
- Computer network connectivity.
- Dependency analysis.
- Database relationship analysis.
- Access control systems.

ADVANTAGES

Floyd's Algorithm

- Finds shortest paths between all pairs of vertices.
- Simple Dynamic Programming approach.
- Works efficiently for dense graphs.
- Can identify indirect shorter paths.

Warshall's Algorithm

- Determines connectivity between all pairs of vertices.
- Simple Boolean operations.
- Easy matrix-based implementation.
- Useful for transitive closure problems.

Common Advantages

- Simple implementation.
- Clear Dynamic Programming formulation.
- Computes all-pairs information.
- Suitable for dense graphs.

LIMITATIONS

- Both algorithms have $O(n^3)$ time complexity.
- Not highly scalable for extremely large graphs.
- Require $O(n^2)$ memory.

- Matrix representation may waste memory for sparse graphs.
- Floyd's Algorithm requires careful handling of infinity values.
- Standard Floyd's Algorithm cannot directly handle certain negative cycle situations without additional checking.

CONCLUSION

Floyd's Algorithm and Warshall's Algorithm were successfully compared using a Dynamic Programming approach. Floyd's Algorithm computes the shortest paths between every pair of vertices, while Warshall's Algorithm determines the transitive closure and reachability between vertices.

Although their objectives and outputs are different, both algorithms follow a highly similar Dynamic Programming formulation. They repeatedly consider each vertex as an intermediate vertex and update a matrix containing the current solution.

Both algorithms have a time complexity of $O(n^3)$ and a space complexity of $O(n^2)$. They are effective for small to medium-sized dense graphs but become computationally expensive for very large graphs.

The comparison demonstrates an important insight into graph-based Dynamic Programming: the same recursive or iterative structure can solve different problems by changing the operation used to update the solution.

REFERENCES

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, Introduction to Algorithms, MIT Press.
2. Richard Bellman, Dynamic Programming, Princeton University Press.
3. Robert W. Floyd, Algorithm 97: Shortest Path, Communications of the ACM.

4. Stephen Warshall, A Theorem on Boolean Matrices, Journal of the ACM.
5. Course Notes on Design and Analysis of Algorithms.